Thread Safety & Concurrency Analysis

Analyzing ctrlSortingComparisson and the OnSwap Event

1. Problem Definition

Thread Safety Level: NONE.

The btnStartCompare_Click method in ctrlSortingComparisson fires off multiple sorting algorithms concurrently using Task.Run. However, the background algorithms communicate back to the UI using a single, static event: Sorting.OnSwap. This bypasses thread safety entirely.

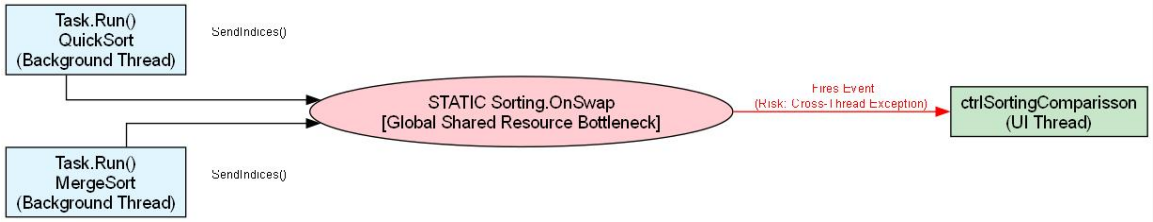
The Weird Bugs Arising From This:

- InvalidOperationException: The background thread fires OnSwap, and the UI subscriber attempts to update the chart on that same background thread, crashing the app.
- Interleaved Chaos: QuickSort and MergeSort run simultaneously. Because OnSwap is global, QuickSort might trigger MergeSort's UI update if the sender string filter fails or lags.

2. Shared Resource Diagram

How OnSwap Sent the Wrong Shared Resource:

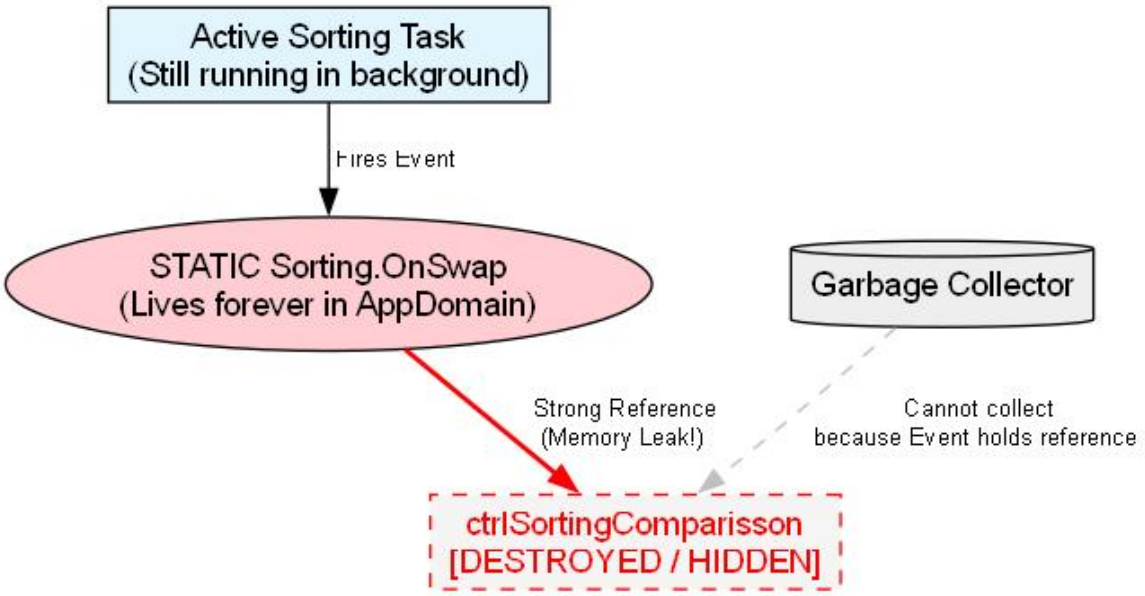
By marking public static event SortingEventHandler OnSwap;, the event delegate list becomes an application-wide shared resource. Instead of each sorting task having its own private channel to the UI, all background tasks dump their swapped indices into a single global bottleneck.



3. Arised Problems: The Back Button Bug

The Memory Leak:

When you click btnBack_Click, you change the control, but you never unsubscribe from Sorting.OnSwap. Because Sorting is a static class, it lives forever in the AppDomain. It holds a strong reference to your destroyed UI control. Future sorts will fire events into dead controls, consuming CPU and RAM until the app crashes.



4. The Wrong Way vs. The Right Way

The Wrong Way:
Using global static events to marshal data from background Tasks to the UI thread. It breaks encapsulation and forces manual thread marshaling (Control.Invoke).
The Right Way:
Using the IProgress<T> interface. It is specifically designed in C# to automatically capture the UI SynchronizationContext and safely marshal background task progress back to the main thread.

5. Solutions to the Concurrency Problem

Solution 1: Adding a Mutex to the Publisher

Where to use it: Inside the Sorting.SendIndices method. You would create a private static readonly object _lock = new object(); and wrap the OnSwap?.Invoke call in a lock(_lock) block.
Drawbacks: Less memory usage, but highly unscalable. If QuickSort needs to update the UI, MergeSort has to pause and wait for the lock. This artificially slows down the algorithms, completely invalidating the performance Metrics you are trying to measure.

Solution 2: Creating Multiple Publisher Objects

Implementation: Remove the static keyword from the Sorting class and the OnSwap event. In the UI, instantiate var quickSorter = new Sorting(); and var mergeSorter = new Sorting();. Subscribe to them separately.
Benefit: Solves the "Shared Resource" bottleneck. Events are strictly isolated to their specific object instances.

Solution 3: Sync using Await and IProgress<T>

Implementation: Pass an IProgress<SortingStateDictionary> into your sorting algorithms instead of using events. Call progress.Report(state) when a swap happens.
Benefit: This is the modern, thread-safe standard. It completely eliminates cross-thread exceptions and memory leaks without needing static events or manual locks.

